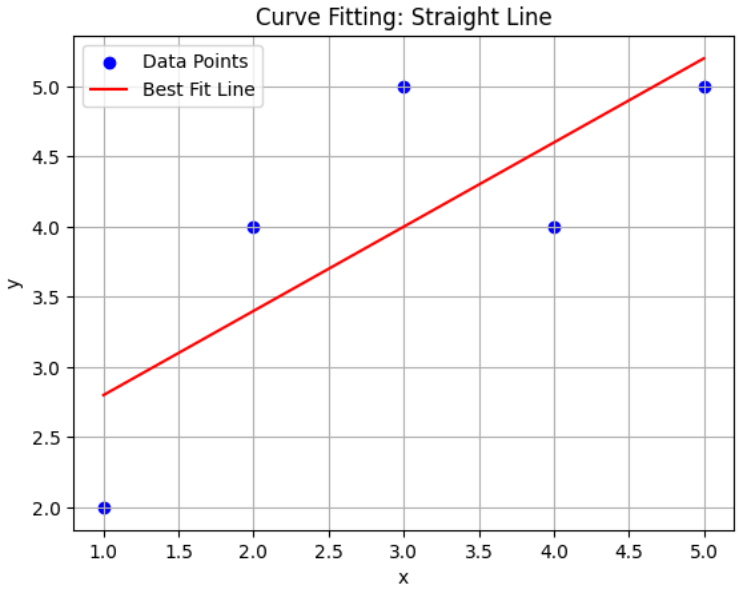
1.Curve Fitting: y=ax+b (Straight Line).

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 5, 4, 5], dtype=float)
n = len(x)
a = (n * np.sum(x * y) - np.sum(x) * np.sum(y)) / (n * np.sum(x**2) -
(np.sum(x))**2)
b = (np.sum(y) - a * np.sum(x)) / n
print("Best Fit Line:")
print(f"y = {a:.4f}x + {b:.4f}")
y_fit = a * x + b
```

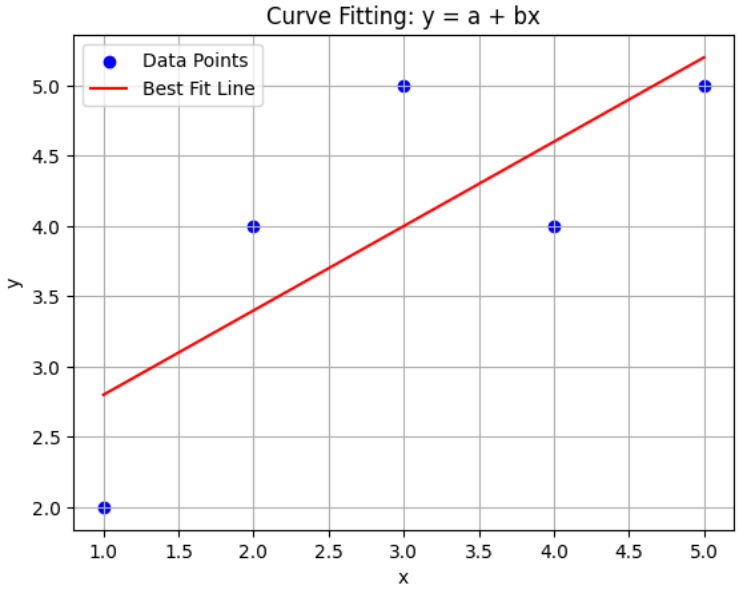



```
plt.scatter(x, y, color='blue', label='Data Points')
plt.plot(x, y_fit, color='red', label='Best Fit Line')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Curve Fitting: Straight Line")
plt.legend()
plt.grid(True)
plt.show()
Best Fit Line:
y = 0.6000x + 2.2000
```

2.Curve Fitting: y=a+bx

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 5, 4, 5], dtype=float)
n = len(x)
b = (n * np.sum(x * y) - np.sum(x) * np.sum(y)) / (n * np.sum(x**2) -
(np.sum(x))**2)
a = (np.sum(y) - b * np.sum(x)) / n
print("Best Fit Line:")
print(f"y = {a:.4f} + {b:.4f}x")
y_fit = a + b * x
plt.scatter(x, y, color='blue', label='Data Points')
plt.plot(x, y_fit, color='red', label='Best Fit Line')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Curve Fitting: y = a + bx")
plt.legend()
plt.grid(True)
plt.show()
```

Best Fit Line:
y = 2.2000 + 0.6000x



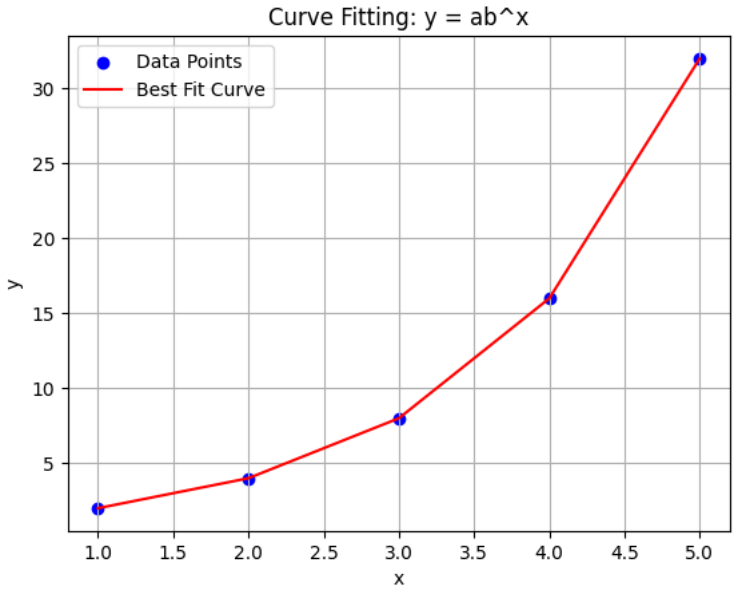
3.Curve Fitting: y=ab^x

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 8, 16, 32], dtype=float)
Y = np.log10(y)
n = len(x)
B = (n * np.sum(x * Y) - np.sum(x) * np.sum(Y)) / (n * np.sum(x**2) -
(np.sum(x))**2)
A = (np.sum(Y) - B * np.sum(x)) / n
a = 10**A
b = 10**B
print("Best Fit Curve:")
print(f"y = {a:.4f} * ({b:.4f})^x")

y_fit = a * (b ** x)

plt.scatter(x, y, color='blue', label='Data Points')
plt.plot(x, y_fit, color='red', label='Best Fit Curve')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Curve Fitting: y = ab^x")
plt.legend()
plt.grid(True)
plt.show()
Best Fit Curve:
y = 1.0000 * (2.0000)^x
```



4.Lagrange Interpolation

```
import numpy as np
x = np.array([1, 2, 3, 4], dtype=float)
y = np.array([1, 4, 9, 16], dtype=float)

xp = float(input("Enter the value of x: "))

n = len(x)
yp = 0
for i in range(n):
p = 1
for j in range(n):
if i != j:
p = p * (xp - x[j]) / (x[i] - x[j])
yp = yp + p * y[i]
print(f"The interpolated value at x = {xp} is {yp:.4f}")
Enter the value of x:  2.5
The interpolated value at x = 2.5 is 6.2500
```

5.Simpson's 1/3 Rule

```
import numpy as np

def f(x):
return x**2
a = float(input("Enter lower limit (a): "))
b = float(input("Enter upper limit (b): "))
n = int(input("Enter number of subintervals (must be even): "))
if n % 2 != 0:
print("Error: Number of subintervals must be even.")
else:
h = (b - a) / n
s = f(a) + f(b)
for i in range(1, n):
x = a + i * h
if i % 2 == 0:
s += 2 * f(x)
else:
s += 4 * f(x)
result = (h / 3) * s
print("Approximate value of the integral =", result)
Enter lower limit (a):  0
Enter upper limit (b):  2
Enter number of subintervals (must be even):  4
Approximate value of the integral = 2.6666666666666665
```

6.Weddle's Rule

```
import numpy as np
def f(x):
return x**2
a = float(input("Enter lower limit (a): "))
b = float(input("Enter upper limit (b): "))
n = int(input("Enter number of subintervals (multiple of 6): "))
if n % 6 != 0:
print("Error: Number of subintervals must be a multiple of 6.")
else:
h = (b - a) / n
result = 0

for i in range(0, n, 6):
x0 = a + i * h
x1 = x0 + h
x2 = x0 + 2 * h
x3 = x0 + 3 * h
x4 = x0 + 4 * h
x5 = x0 + 5 * h
x6 = x0 + 6 * h

result += (3 * h / 10) * (
f(x0) + 5*f(x1) + f(x2) + 6*f(x3) +
f(x4) + 5*f(x5) + f(x6)
)

print("Approximate value of the integral =", result)
```

```
Enter lower limit (a):  0
Enter upper limit (b):  6
Enter number of subintervals (multiple of 6):  6
Approximate value of the integral = 72.0
```

7.Composite Trapezoidal Rule

```
import numpy as np
def f(x):
return x**2
a = float(input("Enter lower limit (a): "))
b = float(input("Enter upper limit (b): "))
n = int(input("Enter number of subintervals: "))
h = (b - a) / n
s = f(a) + f(b)

for i in range(1, n):
x = a + i * h
s += 2 * f(x)

result = (h / 2) * s
print("Approximate value of the integral =", result)
Enter lower limit (a):  0
Enter upper limit (b):  2
Enter number of subintervals:  4
Approximate value of the integral = 2.75
```